

- DA 2009 -

DATA COLLECTION

< // METHODS - II // >

Week 7: Ethical Data Collection & Scraping Challenges

<>// TODAY'S OBJECTIVES //>

- Ethical vs unethical scraping
- Reading and respecting robots.txt
- Scraping vs Crawling
- Browser automation with Selenium
- Introduction to Scrapy
- Anti-scraping techniques & challenges

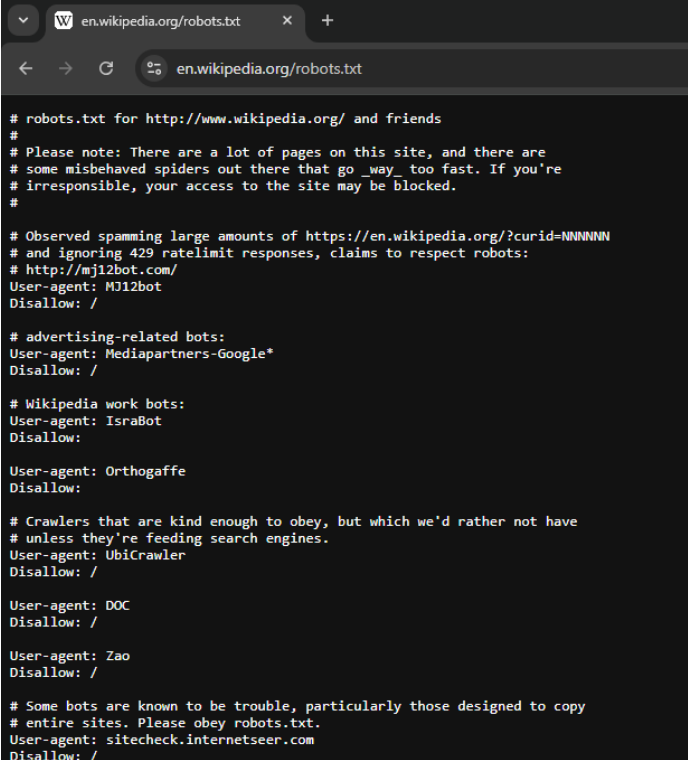


<>// ETHICAL SCRAPING vs UNETHICAL SCRAPING //

Ethical Web Scraping	Unethical Web Scraping
✓ Scrapes only publicly accessible data without bypassing logins, CAPTCHAs, or paywalls.	× Bypasses security (logins, paywalls, CAPTCHAs) to access restricted data.
✓ Respects Terms of Service and follows the site's robots.txt rules.	× Ignores Terms of Service and disobeys robots.txt restrictions.
✓ Uses rate limiting to avoid overloading the server.	× Sends requests too fast, causing performance issues or downtime.
✓ Complies with data privacy laws and avoids scraping personal data without consent.	× Collects personal or sensitive data without permission, risking legal violations.
✓ Credits the data source when sharing results and uses data for fair, lawful purposes.	× Uses scraped data without attribution or for harmful/malicious purposes.

< // WHAT IS ROBOTS.TXT? // >

- **robots.txt** is a text file that tells web crawlers which site pages they can or cannot access.
- It helps website owners manage crawler traffic and protect sensitive areas. Also uses rules for different user-agents to allow or disallow URLs.
- It's a voluntary guideline, not a security tool. Ethical scrapers always respect robots.txt to avoid unwanted scraping and overload.



The screenshot shows a web browser window with the address bar displaying 'en.wikipedia.org/robots.txt'. The page content is a text file with the following rules:

```
# robots.txt for http://www.wikipedia.org/ and friends
#
# Please note: There are a lot of pages on this site, and there are
# some misbehaved spiders out there that go_way_too fast. If you're
# irresponsible, your access to the site may be blocked.
#

# Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
# and ignoring 429 ratelimit responses, claims to respect robots:
# http://mj12bot.com/
User-agent: MJ12bot
Disallow: /

# advertising-related bots:
User-agent: Mediapartners-Google*
Disallow: /

# Wikipedia work bots:
User-agent: IsraBot
Disallow:

User-agent: Orthogaffe
Disallow:

# Crawlers that are kind enough to obey, but which we'd rather not have
# unless they're feeding search engines.
User-agent: UbiCrawler
Disallow: /

User-agent: DOC
Disallow: /

User-agent: Zao
Disallow: /

# Some bots are known to be trouble, particularly those designed to copy
# entire sites. Please obey robots.txt.
User-agent: sitecheck.internetseer.com
Disallow: /
```



WEBSCRAPING vs WEB CRAWLING



Similarities:

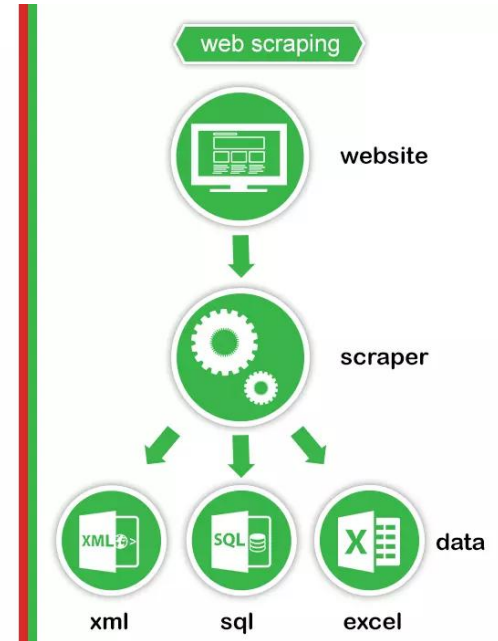
- Both use automated programs to access web pages
- Both parse HTML and extract information
- Both gather web data for analysis or indexing

Differences:

Aspect	Web Scraping	Web Crawling
Purpose	Extract specific data from pages	Systematically browse and index pages
Scope	Targeted pages or data points	Multiple pages, recursive browsing
Output	Structured data (JSON, CSV, etc.)	URL lists or indexes
Frequency	On-demand or periodic	Continuous or scheduled
Focus	Data extraction and cleaning	Link discovery and site traversal

<>// CRAWLERS = SPIDERS? //>

- Web crawlers are called **spiders** or **bots** because they “crawl” the web like a spider moves across its web.
- They start from a set of URLs and follow links across pages, building a network of discovered pages, just like a spider’s web.



< // SELENIUM BROWSER AUTOMATION // >

Selenium is a popular open-source framework that automates web browsers, enabling developers and testers to simulate real user interactions for tasks like website testing, data extraction, and repetitive web actions.

Requirements:

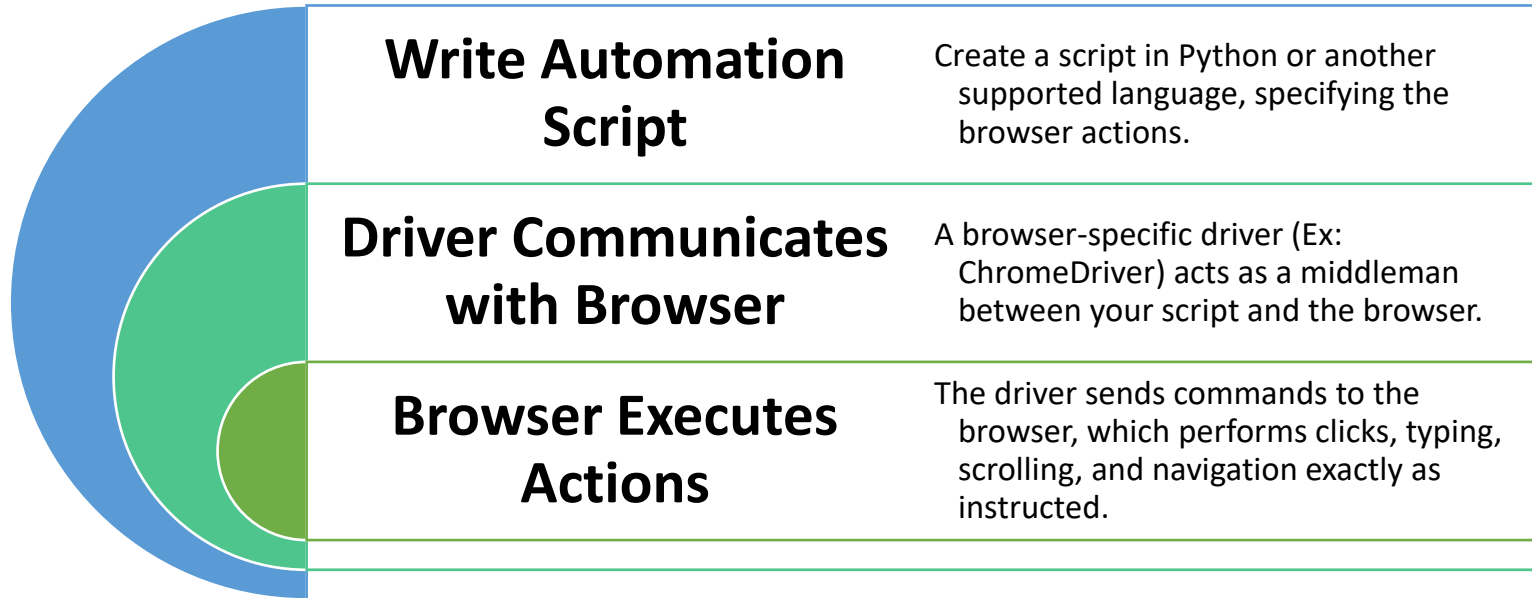
- Install a Supported Web Browser (Ex: Google Chrome, Firefox, Edge).
- Download the Matching Browser Driver (Ex: ChromeDriver for Chrome at <https://googlechromelabs.github.io/chrome-for-testing/>).
- Install Selenium Library (pip install selenium).
- Ensure Browser & Driver Versions Match to avoid compatibility issues.

For more details: <https://www.selenium.dev/>





SELENIUM HOW IT WORKS



< // SELENIUM METHODS AND SYNTAX // >

- **driver.get(url)** - Opens the specified URL in the browser window.
- **driver.maximize_window()** - Maximizes the browser window for better visibility and interaction.
- **driver.find_element(By.ID / NAME / CSS_SELECTOR / XPATH)** - Locates a single web element using different selectors:
 - By.ID: Finds element by its HTML id attribute.
 - By.NAME: Finds element by its name attribute.
 - By.CSS_SELECTOR: Uses CSS selectors for flexible targeting.
 - By.XPATH: Uses XPath expressions to locate elements precisely.

< // SELENIUM METHODS AND SYNTAX // >

- **send_keys(text)** - Sends keystrokes (text input) to input fields, textareas, or editable elements.
- **click()** - Simulates a mouse click on buttons, checkboxes, radio buttons, or links.
- **Select(element).select_by_visible_text(text)** - Selects an option from a dropdown menu by the visible option text.
- **Explicit Waits (WebDriverWait + conditions)** - Waits for a condition (like element clickable or present) before proceeding, improving reliability on dynamic pages.
- **save_screenshot(filename)** - Captures a screenshot of the current browser window and saves it as an image file.



SELENIUM BASIC EXAMPLE



Importing libraries

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
import time
```

Python

Setting up webdriver and checking

```
# Setup: choose Chrome (make sure chromedriver is installed)
driver = webdriver.Chrome()

# 1. Open a test website
driver.get("https://www.selenium.dev/selenium/web/web-form.html")
driver.maximize_window()

# To leave some space between requests
time.sleep(10)

# Quit the driver
driver.quit()
```

Python

< // SCRAPY CRAWLING + SCRAPING // >

Scrapy is a fast, open-source Python framework for web scraping and crawling, designed to efficiently extract data from websites and handle large-scale data collection. This is largely used in industry application due to the project-based nature.

Requirements:

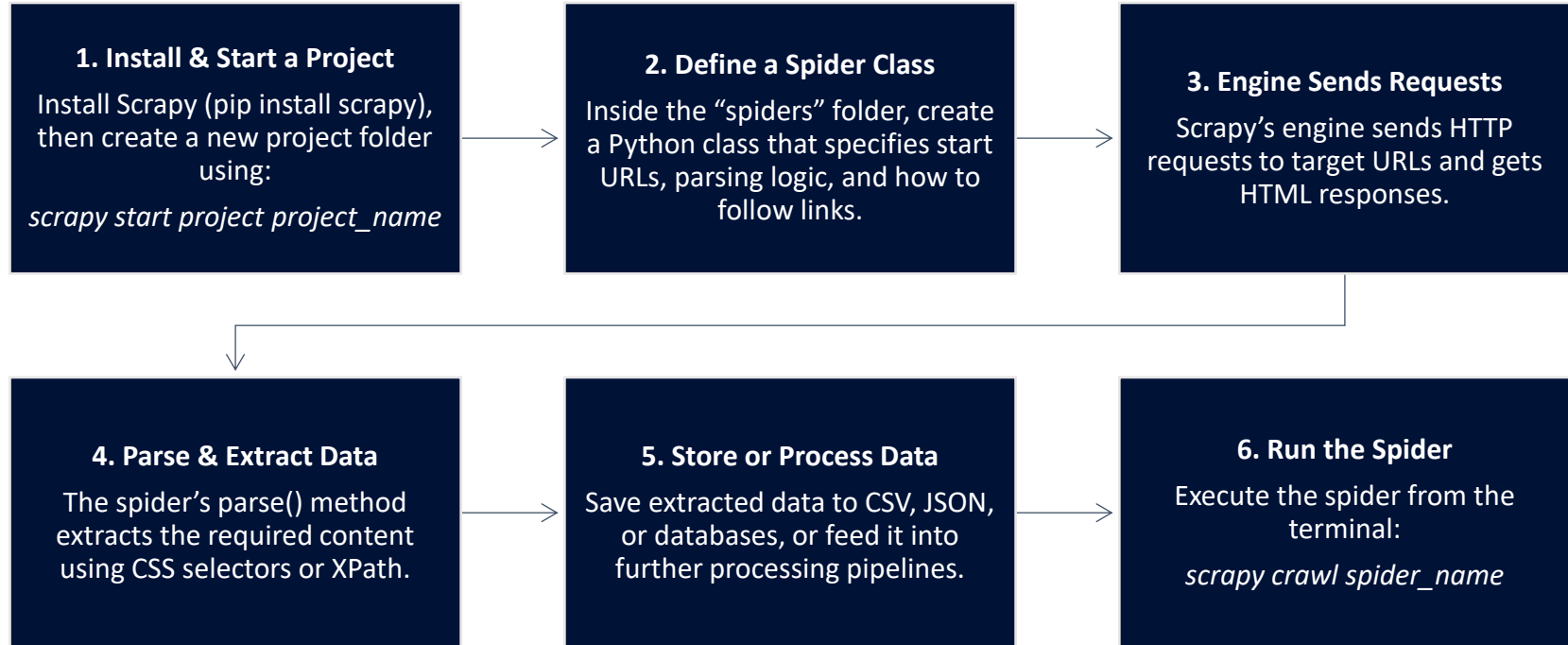
- Install Python (Scrapy works only with Python).
- Install Scrapy Library (pip install scrapy).
- Basic Knowledge of HTML & CSS Selectors to target elements.
- Familiarity with XPath for more precise element selection.
- Knowledge of Object-Oriented Programming and Python Classes



Scrapy



SCRAPY HOW IT WORKS





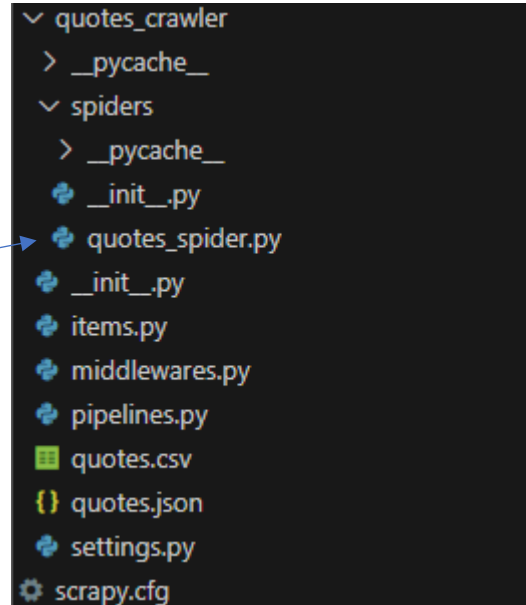
SCRAPY EXAMPLE



Setting up folders (In Terminal):

```
! pip install scrapy  
! scrapy startproject quotes_crawler  
! cd quotes_crawler
```

A folder known as “quotes_crawler” will be automatically created:



- ✓ quotes_crawler
 - > __pycache__
 - ✓ spiders
 - > __pycache__
 - __init__.py
 - quotes_spider.py
 - __init__.py
 - items.py
 - middlewares.py
 - pipelines.py
 - quotes.csv
 - quotes.json
 - settings.py
 - scrapy.cfg

We have to create a file “quotes_spider.py” within the “spiders” folder. This is where we will set up the script.



SCRAPY EXAMPLE



Within the “quotes_spider.py” file:

```
import scrapy

You, 13 hours ago | 1 author (You)
class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ["http://quotes.toscrape.com"]

    def parse(self, response):
        # Loop through all quotes on the page
        for quote in response.css("div.quote"):
            yield {
                'text': quote.css("span.text::text").get(),
                'author': quote.css("small.author::text").get(),
                'tags': quote.css("div.tags a.tag::text").getall()
            }

        # Follow pagination link
        next_page = response.css("li.next a::attr(href)").get()
        if next_page:
            yield response.follow(next_page, self.parse)
```

To extract outputs:

To simply run the crawler and scrape the site

```
# !scrapy crawl quotes
```

To run the crawler and extract as a json file

```
# !scrapy crawl quotes -o quotes.json
```

Similarly if you want a csv

```
# !scrapy crawl quotes -o quotes.csv
```

<>// ANTI-SCRAPING & SCRAPING CHALLENGES //

- Common anti-scraping techniques help websites protect their data and servers

Technique	Purpose	Good (Benefits)	Bad (Challenges)
Headers & User-Agent Checks	Detect bots by checking request headers	Easy to implement, blocks simple bots	Easy to spoof, low protection
Rate Limiting	Limit number of requests per IP/user	Prevents server overload	May block legitimate users, proxy evasion
CAPTCHAs	Verify human interaction	Highly effective against bots	Hurts user experience, can be bypassed
IP Blocking / Blacklisting	Block suspicious IP addresses	Stops known bad actors	Dynamic IPs and proxies reduce effectiveness



USING PROPER HEADERS



- Many websites block requests with no or default User-Agent headers (like Python-requests default).
- Setting a realistic User-Agent mimics a real browser, reducing the chance of blocks.
- This helps avoid being mistaken for a malicious bot and can prevent your scraper from being blocked.
- It's respectful and helps web servers identify traffic sources better.

Example of Using Proper Headers in a Scraping Request (Python with requests):

```
import requests

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                 "AppleWebKit/537.36 (KHTML, like Gecko) "
                 "Chrome/115.0 Safari/537.36"
}

response = requests.get("https://example.com", headers=headers)
print(response.status_code)
```

✓ 2.1s

200

<>// WEB SCRAPING BEST PRACTICES //

- **Respect Website Policies** – Check and follow the site's robots.txt file and terms of service.
- **Rate Limiting & Throttling** – Add delays between requests to avoid overloading servers and getting blocked.
- **Avoid Over-Requesting** – Only scrape the data you need; don't repeatedly fetch unchanged content.
- **Use Proper Headers** – Include a meaningful User-Agent to identify your crawler responsibly.
- **Handle Errors Gracefully** – Anticipate 404/500 responses, CAPTCHAs, and changes in site structure.
- **Store Data Efficiently** – Save in structured formats (CSV, JSON, database) and back up regularly.
- **Test Selectors Regularly** – HTML structure can change; keep your scraping logic up to date.

< // TO DO LIST BEFORE WEEK 8 // >

- Go through group assignment details and start working on an interesting project!
- Learn more about Selenium & Scrapy and try it on more websites.
- Get comfortable with the ethics behind scraping





THANK YOU

<SEE YOU NEXT WEEK!>

Susara Ouchithya
susara.ouchithya@gmail.com